

# CS-338 Intro to the Theory of Computation

- Lecture #11

## ➤ Mapping Reducibility

- Prof Pietro S. Oliveto  
Department of Computer Science and Engineering  
Southern University of Science and Technology (SUSTech)
- `olivetop@sustech.edu.cn`  
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

# Overview

## Last time:

- $A_{TM}$  is undecidable
- The diagonalization method
- $\overline{A_{TM}}$  is T-unrecognizable
- The **Reducibility Method** for proving undecidability and T-unrecognizability
- **General Reducibility:**  $HALT_{TM}$ ,  $E_{TM}$ ,  $EQ_{TM}$ ,  $REGULAR_{TM}$  are undecidable
- **Rice's Theorem:** All non trivial properties of recognizable languages are undecidable

## Today: (Sipser §5.1, §5.3)

- Mapping reducibility

# Reducibility – Concept Overview

If we have two languages (or problems)  $A$  and  $B$ , then  $A$  is reducible to  $B$  means that we can use  $B$  to solve  $A$ .

**Example 4:** We showed that  $A_{\text{NFA}}$  is reducible to  $A_{\text{DFA}}$ .

If  $A$  is reducible to  $B$  then solving  $B$  gives a solution to  $A$ .

- then  $B$  is easy  $\rightarrow A$  is easy.

- then  $A$  is hard  $\rightarrow B$  is hard.

this is the form we will use

General strategy

- $A_{\text{TM}}$  proved undecidable via diagonalization
- All other problems can be shown undecidable by reducing some undecidable problem to them!

# Mapping Reducibility

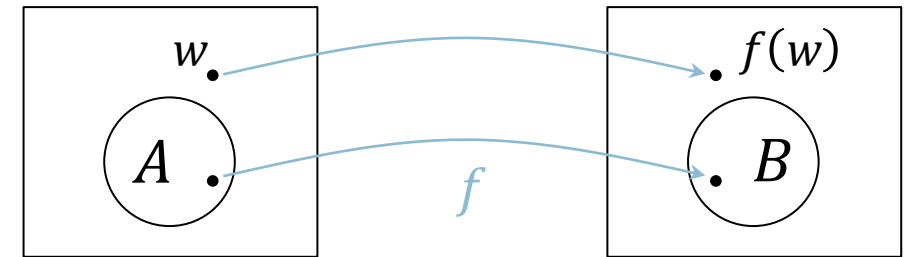
- We have used the notion of **reducibility** to prove several languages undecidable: now we formalise it by specialising it
- A formal notion allows us to use reducibility:
  1. In more refined ways to prove languages not Turing-recognizable
  2. For applications in complexity theory

**Idea:** A problem  $A$  is reduced to problem  $B$  by using **mapping reducibility** if there exists a computable function  $f$  that converts instances of a problem  $A$  into instances of a problem  $B$ .

**Defn:** Function  $f: \Sigma^* \rightarrow \Sigma^*$  is computable if there is a TM  $F$  where  $F$  on input  $w$  halts with  $f(w)$  on its tape, for all strings  $w$ .

A TM computes a function by starting with the input to the function on the tape and halting with the output of the function on the tape.

**Defn:** Language  $A$  is mapping-reducible to language  $B$  ( $A \leq_m B$ ) if there is a computable function  $f: \Sigma^* \rightarrow \Sigma^*$  where  $w \in A$  iff  $f(w) \in B$ . The function  $f$  is called the **reduction** from  $A$  to  $B$ .



- If we have such a **reduction** we can solve  $A$  with a solver for  $B$   
(Because any instance of  $A$  can be solved by first using the reduction to convert it to an instance of  $B$ , and then applying the solver for  $B$ )

# Mapping Reductions - properties

- To test whether  $w \in A$ , we use the reduction  $f$  to map  $w$  to  $f(w)$  and test whether  $f(w) \in B$
- If one problem is mapping reducible to a second, previously solved problem, we can thereby obtain a solution to the original problem

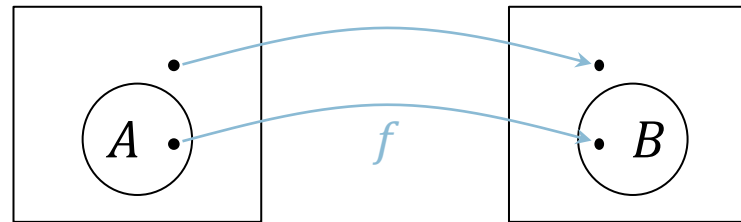
**Theorem:** If  $A \leq_m B$  and  $B$  is decidable then so is  $A$

Proof: Say TM  $R$  decides  $B$ .

Construct TM  $S$  deciding  $A$ :

$S$  = "On input  $w$

1. Compute  $f(w)$
2. Run  $R$  on  $f(w)$  to test if  $f(w) \in B$
3. If  $R$  halts then output same result."



## Check-in 9.2

Suppose  $A \leq_m B$ .

What can we conclude?

Check all that apply.

- (a)  $B \leq_m A$
- (b)  $\overline{A} \leq_m \overline{B}$
- (c) None of the above

**Corollary:** If  $A \leq_m B$  and  $A$  is undecidable then so is  $B$

**Theorem:** If  $A \leq_m B$  and  $B$  is T-recognizable then so is  $A$

Proof: Same as above (except that  $R$  and  $S$  are now recognisers).

**Corollary:** If  $A \leq_m B$  and  $A$  is T-unrecognizable then so is  $B$

# Example: $HALT_{TM}$ is undecidable

- Let's mapping reduce  $A_{TM}$  to  $HALT_{TM}$
- We must present a computable function  $f: \Sigma^* \rightarrow \Sigma^*$  that takes an input of the form  $\langle M, w \rangle$  and returns an output of the form  $\langle M', w' \rangle$  where

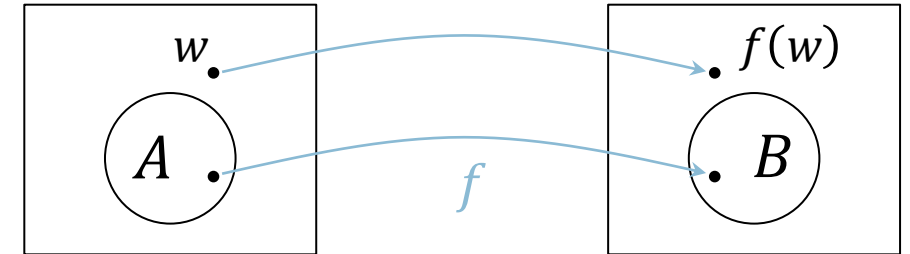
$$\langle M, w \rangle \in A_{TM} \iff \langle M', w' \rangle \in HALT_{TM}$$

The following TM  $F$  computes the reduction  $f$

$F =$  "On input  $\langle M, w \rangle$

- Construct a new TM  $M' =$  "On input  $x$ 
  - Run  $M$  on  $x$ .
  - If  $M$  accepts, *accept*
  - If  $M$  rejects, enter a loop."
- Output  $\langle M', w \rangle$ "

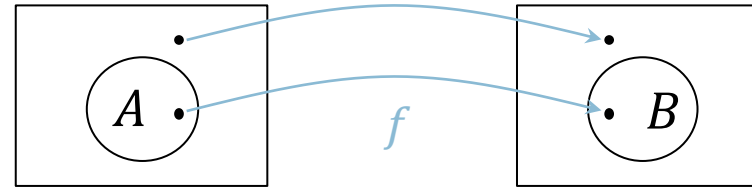
- If the input to  $F$  is not proper, then  $F$  returns a string not in  $HALT_{TM}$



# Mapping vs General Reducibility

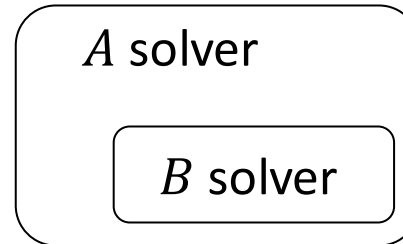
Mapping Reducibility of  $A$  to  $B$ : Translate  $A$ -questions to  $B$ -questions.

- A special type of reducibility
- Useful to prove T-unrecognizability



(General) Reducibility of  $A$  to  $B$ : Use  $B$  solver to solve  $A$ .

- May be conceptually simpler
- Useful to prove undecidability



Noteworthy difference:

- $A$  is reducible to  $\overline{A}$
- $A$  may not be mapping reducible to  $\overline{A}$ .

For example  $\overline{A_{TM}} \not\leq_m A_{TM}$

## Check-in 9.3

We showed that if  $A \leq_m B$  and  $B$  is T-recognizable then so is  $A$ .

Is the same true if we use general reducibility instead of mapping reducibility?

- (a) Yes
- (b) No

Check-in 9.3

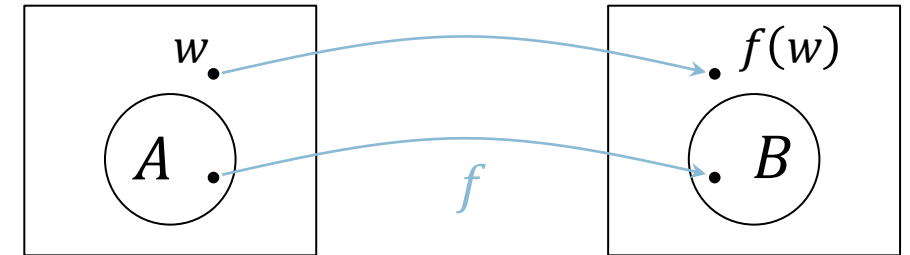
# $\overline{E_{TM}}$ is undecidable: general and mapping notion difference example

- Let's try to mapping reduce  $A_{TM}$  to  $E_{TM}$

Recall

$S =$  "On input  $\langle M, w \rangle$

- Transform  $M$  to new TM  $M_w =$  "On input  $x$ 
  - If  $x \neq w$ , *reject*.
  - else run  $M$  on  $w$
  - Accept* if  $M$  accepts."
- Use  $R$  to test whether  $L(M_w) = \emptyset$
- If YES [so  $M$  rejects  $w$ ] then *reject*.  
If NO [so  $M$  accepts  $w$ ] then *accept*.



$$\text{So } L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ rejects } w \end{cases}$$

- $M$  accepts  $w$  iff  $L(M_w)$  is **not** empty
- $f(\langle M, w \rangle) = \langle M_w \rangle$  is **not** a mapping reduction from  $A_{TM}$  to  $E_{TM}$ :  $\langle M, w \rangle \in A_{TM}$  &  $\langle M_w \rangle \notin E_{TM}$
- The reduction of the proof works but it is not a mapping reduction.

**However:**  $A_{TM} \leq_m \overline{E_{TM}}$

The computable reduction function  $f$  is  $f(\langle M, w \rangle) = \langle M_w \rangle$

Because  $\langle M, w \rangle \in A_{TM}$  iff  $\langle M_w \rangle \in \overline{E_{TM}}$   
(  $M$  accepts  $w$  iff  $L(\langle M_w \rangle) \neq \emptyset$  )



# Reducibility – Templates

To prove  $B$  is undecidable:

- Show undecidable  $A$  is reducible to  $B$ . (often  $A$  is  $A_{\text{TM}}$ )
- Template: Assume TM  $R$  decides  $B$ .  
Construct TM  $S$  deciding  $A$ . Contradiction.

To prove  $B$  is T-unrecognizable:

- Show T-unrecognizable  $A$  is mapping reducible to  $B$ . (often  $A$  is  $\overline{A_{\text{TM}}}$ )
- Template: give reduction function  $f$ .

# $E_{\text{TM}}$ is T-unrecognizable

Recall  $E_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset \}$

**Theorem:**  $E_{\text{TM}}$  is T-unrecognizable

Proof: Show  $\overline{A_{\text{TM}}} \leq_m E_{\text{TM}}$

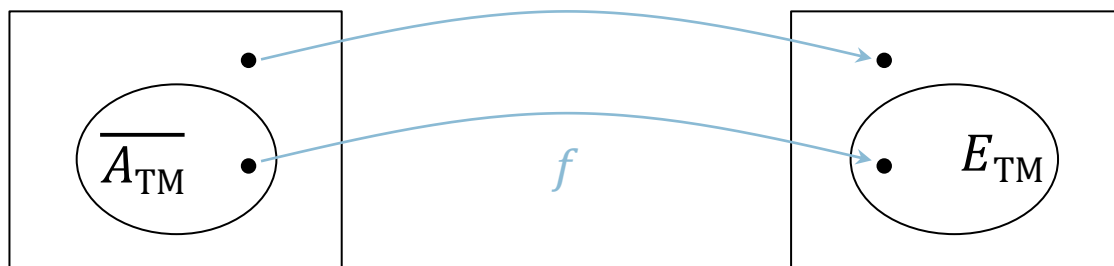
Reduction function:  $f(\langle M, w \rangle) = \langle M_w \rangle$       Recall TM  $M_w =$  “On input  $x$

Explanation:  $\langle M, w \rangle \in \overline{A_{\text{TM}}}$  iff  $\langle M_w \rangle \in E_{\text{TM}}$

$M$  rejects  $w$  iff  $L(\langle M_w \rangle) = \emptyset$

1. If  $x \neq w$ , *reject*.
2. else run  $M$  on  $w$
3. *Accept* if  $M$  accepts.”

So  $L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ rejects } w \end{cases}$



# $EQ_{TM}$ and $\overline{EQ_{TM}}$ are T-unrecognizable

$$EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are TMs and } L(M_1) = L(M_2) \}$$

**Theorem:** Both  $EQ_{TM}$  and  $\overline{EQ_{TM}}$  are T-unrecognizable

Proof: (1)  $\overline{A_{TM}} \leq_m EQ_{TM}$   
(2)  $A_{TM} \leq_m \overline{EQ_{TM}}$

For any  $w$  let  $T_w =$  “On input  $x$   $T_w$  acts on all inputs the way  $M$  acts on  $w$ .

1. Ignore  $x$ .
2. Simulate  $M$  on  $w$ .”

(1) Here we give  $f$  which maps  $\overline{A_{TM}}$  problems (of the form  $\langle M, w \rangle$ ) to  $EQ_{TM}$  problems (of the form  $\langle T_1, T_2 \rangle$ ).

$$f_1(\langle M, w \rangle) = \langle T_w, T_{\text{reject}} \rangle \quad T_{\text{reject}} \text{ is a TM that always rejects.}$$

(2) Similarly  $f_2(\langle M, w \rangle) = \langle T_w, T_{\text{accept}} \rangle \quad T_{\text{accept}}$  always accepts.

# Reducibility terminology

## Why do we use the term “reduce”?

When we reduce  $A$  to  $B$ , we show how to solve  $A$  by using  $B$  and conclude that  $A$  is no harder than  $B$ . (suggests the  $\leq_m$  notation)

Possibility 1: We bring  $A$ 's difficulty down to  $B$ 's difficulty.

Possibility 2: We bring  $B$ 's difficulty up to  $A$ 's difficulty.

# Quick review of today

1. Defined mapping reducibility as a type of reducibility.
2.  $E_{\text{TM}}$  is T-unrecognizable.
3.  $EQ_{\text{TM}}$  and  $\overline{EQ_{\text{TM}}}$  are T-unrecognizable.